Kubernetes and Cloud Native Associate - KCNA

Kubernetes Resources

Demo ReplicaSets

In this lesson, you'll learn how to create and manage a ReplicaSet using a pre-existing Pod definition file. Previously, you created a Pod using YAML; now, we'll extend that knowledge by grouping Pods into a ReplicaSet to ensure that a consistent number of identical Pods is running at all times.

Creating the ReplicaSet YAML

Start by navigating to your project directory. You should already have a directory named `pods` containing your Pod definition files. Next, create a new directory for ReplicaSets and add a file called `replicaset.yaml` inside it.

Open `replicaset.yaml` and begin by specifying the API version and kind. For ReplicaSets, use `apps/v1` as the API version and `ReplicaSet` as the kind. Then add metadata including the ReplicaSet's name and labels. In this example, we use the label `app: myapp`.

Under the `spec` section:

- Define the `selector` that matches the labels on the Pods managed by this ReplicaSet.

- Set the number of replicas (e.g., 3).
- Provide the Pod template. You can copy the template from your existing Pod definition, but ensure that the indentation is correct after pasting.

Matching Labels

Ensure that the labels under the `selector` and in the Pod template exactly match, as these are the only labels that affect the ReplicaSet's operation. The label assigned to the ReplicaSet itself in the metadata is not used for matching.

Below is an example ReplicaSet definition:

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
spec:
  selector:
    matchLabels:
      env: production
  replicas: 3
  template:
    metadata:
      name: nginx-2
      labels:
        env: production
    spec:
      containers:
        - name: nginx
          image: nginx
---
apiVersion: v1
```

```
kind: Pod
metadata:
  name: nginx-2
  labels:
    env: production
spec:
  containers:
    - name: nginx
      image: nginx
```

Validating the ReplicaSet Configuration

After saving the file, verify that your directory structure is correct. Your project root should now contain a new directory (for example, `**replicatesets**`) with the `**replicaset.yaml**` file. For instance:

```
admin@ubuntu-server kubernetes-for-beginners # ls
pods replicates
admin@ubuntu-server kubernetes-for-beginners # cd replicatesets/
```

Check the file by listing its contents:

```
admin@ubuntu-server replicatesets # ls
replicaset.yaml
admin@ubuntu-server replicatesets #
```

Clear your screen and create the ReplicaSet using:

```
kubectl create -f replicatesets/replicaset.yaml
```

Soon after, you can verify the ReplicaSet status with:

```
kubectl get replicaset
```

Expected output:

```
NAME              DESIRED   CURRENT   READY   AGE
myapp-replicaset   3         3         3       8s
```

This output confirms that the ReplicaSet has successfully created three Pods. To inspect the created Pods further, run:

```
kubectl get pods
```

You might see output similar to:

```
NAME                    READY   STATUS    RESTARTS   AGE
myapp-replicaset-8nxxl   1/1     Running   0          24s
myapp-replicaset-jlgr2   1/1     Running   0          24s
myapp-replicaset-pm4rl   1/1     Running   0          24s
```

Note how each Pod's name begins with `**myapp-replicaset**`, indicating its association with the ReplicaSet.

Testing the ReplicaSet Self-Healing

ReplicaSets continuously ensure that the defined number of Pods is running. To test this self-healing mechanism:

1. **List Your Pods:**
   Identify a Pod to delete (e.g., one ending with `**8nxxl**`):

```
kubectl get pods
```

Sample output:

```
NAME                    READY   STATUS    RESTARTS   AGE
myapp-replicaset-8nxxl  1/1     Running   0          45s
myapp-replicaset-jlgr2  1/1     Running   0          45s
myapp-replicaset-pm4rl  1/1     Running   0          45s
```

2. **Delete the Selected Pod:**

```
kubectl delete pod myapp-replicaset-8nxxl
```

You should see confirmation similar to:

```
pod "myapp-replicaset-8nxxl" deleted
```

After a few seconds, list the Pods again. The ReplicaSet will have automatically created a new Pod to maintain the desired replica count. To inspect the ReplicaSet details, including its events, run:

```
kubectl describe replicaset myapp-replicaset
```

Scroll through the details to verify that a new Pod was created after deletion.

Preventing Extra Pods from Running

A core feature of ReplicaSets is to ensure that only the specified number of Pods are active. If you attempt to create an additional Pod with a label matching the ReplicaSet selector, the ReplicaSet controller will automatically delete the extra Pod.

For instance, update your existing `**nginx.yaml**` file so that the Pod uses the same label as defined in the ReplicaSet selector:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-2
  labels:
    app: myapp
spec:
  containers:
    - name: nginx
      image: nginx
```

Before applying changes, verify the current Pods:

```
kubectl get pods
```

Then, create the Pod:

```
kubectl create -f nginx.yaml
```

Shortly after, run:

```
kubectl get pods
```

You'll notice that the extra Pod is immediately marked for termination. The ReplicaSet controller consistently enforces that only the predefined number of Pods remain active. To see related events, describe the ReplicaSet:

```
kubectl describe replicaset myapp-replicaset
```

Updating the ReplicaSet

There are scenarios where you may need to adjust the number of replicas in your ReplicaSet. There are two methods to achieve this:

Method 1: Edit the Running Configuration

Use `kubectl edit` to modify the live configuration of the ReplicaSet. This command opens the configuration in your default text editor (such as Vim):

```
kubectl edit replicaset myapp-replicaset
```

Modify the `spec.replicas` field (for example, change it from 3 to 4), then save and exit the editor. Kubernetes will immediately update the ReplicaSet and create new Pods as necessary. Verify the update by listing your Pods:

```
kubectl get pods
```

Method 2: Utilize the Scale Command

Alternatively, you can scale the ReplicaSet directly:

```
kubectl scale replicaset myapp-replicaset --replicas=2
```

This command scales the ReplicaSet down to two Pods. Verify the change with:

```
kubectl get pods
```

You may see an output similar to:

```
NAME                   READY   STATUS      RESTARTS   AGE
myapp-replicaset-bvlst  0/1     Terminating 0          5m30s
myapp-replicaset-cssz8  0/1     Terminating 0          48s
```

```
myapp-replicaset-jlgr2  1/1     Running     0          6m31s
myapp-replicaset-pm4rl  1/1     Running     0          6m31s
```

After termination completes, only two Pods will remain active.

Conclusion

In this lesson, you learned how to create a ReplicaSet from an existing Pod definition file, observed the self-healing behavior when Pods are deleted, and explored two methods to update the number of replicas—editing the running configuration or scaling directly. These features ensure that your cluster consistently maintains the desired number of active Pods.

Happy clustering, and see you in the next lesson!

Watch Video

Watch video content

Previous

ReplicaSets

Next

Deployments